

Skalerbare Design Systems i Frontend

En praktisk guide til at bygge, strukturere og vedligeholde komponentbiblioteker med Radix UI, compound components og design tokens.

Indhold

1. Hvad er et design system?
2. Arkitektur og mappestruktur
3. Design tokens
4. Compound components pattern
5. Headless UI med Radix
6. Wrapping Radix i dit design system
7. Styling-strategier
8. Dokumentation med Storybook
9. Versionering og distribution
10. Governance og skalering på tværs af teams
11. Anti-patterns at undgå
12. Checkliste

1. Hvad er et design system?

Et design system er mere end et komponentbibliotek. Det er en samling af genanvendelige komponenter, design tokens, retningslinjer og værktøjer der sikrer konsistens på tværs af produkter og teams.

Et design system består typisk af:

- **Design tokens** — farver, spacing, typografi, shadows defineret centralt
- **Komponentbibliotek** — genbrugelige UI-komponenter (Button, Select, Modal osv.)
- **Dokumentation** — Storybook, usage guidelines, do's and don'ts
- **Governance** — hvem bidrager, review-proces, breaking changes

Forskellen mellem et godt og et dårligt design system ligger ofte i komponent-API design. Et dårligt API ender med 30+ props per komponent. Et godt API bruger composition og giver udviklere kontrol uden kompleksitet.

2. Arkitektur og mappestruktur

En velstruktureret mappestruktur er fundamentet. Hvert komponent lever i sin egen mappe med alt hvad det behøver:

```
packages/ui/  
src/  
tokens/  
colors.ts  
spacing.ts  
typography.ts  
index.ts  
components/  
Button/  
Button.tsx  
Button.styles.ts  
Button.test.tsx  
Button.stories.tsx  
index.ts  
Select/  
SelectRoot.tsx  
SelectTrigger.tsx  
SelectContent.tsx  
SelectItem.tsx  
SelectValue.tsx  
Select.styles.ts  
Select.test.tsx  
Select.stories.tsx  
index.ts  
index.ts # Public API - alt eksporteres herfra
```

Tip: Hold altid en enkelt `index.ts` i roden af dit library som den eneste public API. Alt andet er implementation details.

3. Design tokens

Design tokens er de primitive værdier der definerer dit visuelle sprog. De fungerer som kontrakten mellem design og kode.

Primitive tokens

```
// tokens/colors.ts
export const colors = {
  // Primitive scale
  indigo50: '#eef2ff',
  indigo100: '#e0e7ff',
  indigo500: '#6366f1',
  indigo600: '#4f46e5',
  indigo700: '#4338ca',

  gray50: '#f8fafe',
  gray100: '#f1f5f9',
  gray200: '#e2e8f0',
  gray700: '#334155',
  gray900: '#0f172a',
} as const;
```

Semantiske tokens

```
// tokens/semantic.ts
import { colors } from './colors';

export const semantic = {
  // Semantiske aliases
  colorPrimary: colors.indigo600,
  colorPrimaryHover: colors.indigo700,
  colorTextDefault: colors.gray900,
  colorTextMuted: colors.gray700,
  colorBgSurface: colors.gray50,
  colorBorder: colors.gray200,

  // Spacing scale
  space1: '4px',
  space2: '8px',
  space3: '12px',
  space4: '16px',
  space6: '24px',
  space8: '32px',

  // Border radius
  radiusSm: '4px',
  radiusMd: '8px',
```

```
radiusLg: '12px',  
radiusFull: '9999px',  
} as const;
```

Tip: Primitive tokens (indigo500) er stabile og ændres sjældent. Semantiske tokens (colorPrimary) mapper til primitives og kan ændres uden at røre komponenterne. Dette er nøglen til theming.

4. Compound components pattern

Det vigtigste arkitekturvalg i et skalerbart design system er at bruge **composition over configuration**. I stedet for at løse alle use cases med props, lader du brugeren sammensætte delkomponenter.

Problem: Prop explosion

```
// Anti-pattern: alt styres via props
<Select
  label="Land"
  placeholder="Vaelg..."
  options={options}
  isSearchable
  isMulti
  isClearable
  isDisabled={false}
  menuPlacement="top"
  onChange={...}
  customFilter={...}
  renderOption={...}
  renderValue={...}
  noOptionsMessage="Ingen resultater"
  loadingMessage="Henter..."
// ... 20 props mere
/>
```

Løsningen: Compound components

```
// Composition: hver del er sit eget komponent
<Select.Root>
  <Select.Trigger>
    <Select.Value placeholder="Vaelg..." />
  </Select.Trigger>
  <Select.Content>
    <Select.Group>
      <Select.Label>Europa</Select.Label>
      <Select.Item value="dk">Danmark</Select.Item>
      <Select.Item value="se">Sverige</Select.Item>
    </Select.Group>
  </Select.Content>
</Select.Root>
```

Eksportering via namespace

```
// Select/index.ts
import { SelectRoot } from './SelectRoot';
import { SelectTrigger } from './SelectTrigger';
import { SelectValue } from './SelectValue';
import { SelectContent } from './SelectContent';
import { SelectItem } from './SelectItem';
import { SelectGroup } from './SelectGroup';
```

```
import { SelectLabel } from './SelectLabel';

export const Select = {
  Root: SelectRoot,
  Trigger: SelectTrigger,
  Value: SelectValue,
  Content: SelectContent,
  Item: SelectItem,
  Group: SelectGroup,
  Label: SelectLabel,
};
```

Forbrugeren importerer `import { Select } from '@myorg/ui'` og får fuld autocompletion på `Select`. — rent, discoverable og typesafe.

Advarsel: Undgå det ældre mutation-pattern (`widget.Image = widgetImage`) — det fungerer, men TypeScript inferer typerne dårligere og det muterer det originale objekt.

5. Headless UI med Radix

Radix UI er et headless komponentbibliotek der leverer behavior og accessibility uden styling. Det håndterer de svære ting: keyboard navigation, focus management, ARIA attributter, scroll locking, portal rendering osv.

Hvad Radix giver dig:

- **Accessibility out of the box** — korrekte ARIA roles, keyboard nav, focus trapping
- **Ingen styling** — du har fuld kontrol over udseendet
- **Compound component API** — composition-first arkitektur
- **Controlled og uncontrolled** — begge modes supported
- **Animation support** — data-attributter til entry/exit animationer

Direkte brug af Radix:

```
import * as RadixSelect from '@radix-ui/react-select';

// Radix eksporterer named exports
// Du bruger dem med namespace import
<RadixSelect.Root>
  <RadixSelect.Trigger>
    <RadixSelect.Value placeholder="Vaelg..." />
  </RadixSelect.Trigger>
  <RadixSelect.Portal>
    <RadixSelect.Content>
      <RadixSelect.Viewport>
        <RadixSelect.Item value="dk">
          <RadixSelect.ItemText>Danmark</RadixSelect.ItemText>
          <RadixSelect.ItemIndicator>
            <CheckIcon />
          </RadixSelect.ItemIndicator>
        </RadixSelect.Item>
      </RadixSelect.Viewport>
    </RadixSelect.Content>
  </RadixSelect.Portal>
</RadixSelect.Root>
```

Bemærk: Radix's rå API er bevidst verbose — det eksponerer alle primitiver. Dit design system's opgave er at wrappe dette i en simplere API.

6. Wrapping Radix i dit design system

Den vigtigste pointe: forbrugere af dit design system importerer **aldrig** direkte fra Radix. De importerer fra jeres package. Det giver dig tre fordele:

- **Kontrol over API'et** — du kan forenkle, begrænse og tilpasse
- **Konsistent styling** — tokens og styles er bagt ind
- **Udskiftelighed** — du kan bytte Radix ud uden breaking changes

Eksempel: SelectTrigger wrapper

```
// components/Select/SelectTrigger.tsx
import * as RadixSelect from '@radix-ui/react-select';
import { forwardRef } from 'react';
import { cn } from '../../utils/cn';
import styles from './Select.module.css';

interface SelectTriggerProps {
  children: React.ReactNode;
  size?: 'sm' | 'md' | 'lg';
  variant?: 'default' | 'ghost';
  className?: string;
}

export const SelectTrigger = forwardRef<
  HTMLButtonElement,
  SelectTriggerProps
>(({ children, size = 'md', variant = 'default', className, ...props }, ref) => (
  <RadixSelect.Trigger
    ref={ref}
    className={cn(
      styles.trigger,
      styles[size],
      styles[variant],
      className
    )}
    {...props}
  >
    {children}
  </RadixSelect.Trigger>
));

SelectTrigger.displayName = 'SelectTrigger';
```

Komplet Select wrapper

```
// components/Select/index.ts
import { SelectRoot } from './SelectRoot';
import { SelectTrigger } from './SelectTrigger';
import { SelectValue } from './SelectValue';
import { SelectContent } from './SelectContent';
import { SelectItem } from './SelectItem';

export const Select = {
  Root: SelectRoot,
  Trigger: SelectTrigger,
  Value: SelectValue,
  Content: SelectContent,
  Item: SelectItem,
};

// Eksporter types separat
export type { SelectRootProps } from './SelectRoot';
export type { SelectTriggerProps } from './SelectTrigger';
export type { SelectItemProps } from './SelectItem';
```

Brug i applikationen:

```
// I en feature-komponent
import { Select } from '@azets/ui';

function CountryPicker() {
  return (
    <Select.Root>
      <Select.Trigger size="md">
        <Select.Value placeholder="Vaelg land" />
      </Select.Trigger>
      <Select.Content>
        <Select.Item value="dk">Danmark</Select.Item>
        <Select.Item value="se">Sverige</Select.Item>
        <Select.Item value="no">Norge</Select.Item>
      </Select.Content>
    </Select.Root>
  );
}
```

Tip: Brug `forwardRef` på alle wrapper-komponenter så refs propagerer korrekt. Dette er kritisk for form libraries og focus management.

7. Styling-strategier

Der er tre gængse tilgange:

CSS Modules

```

/* Select.module.css */
.trigger {
display: inline-flex;
align-items: center;
border: 1px solid var(--color-border);
border-radius: var(--radius-md);
background: var(--color-bg-surface);
}

.trigger[data-state='open'] {
border-color: var(--color-primary);
}

.sm { height: 32px; padding: 0 8px; font-size: 13px; }
.md { height: 40px; padding: 0 12px; font-size: 14px; }
.lg { height: 48px; padding: 0 16px; font-size: 16px; }

```

Tailwind + CVA (Class Variance Authority)

```

import { cva } from 'class-variance-authority';

const triggerVariants = cva(
// Base classes
'inline-flex items-center border rounded-md bg-white',
{
variants: {
size: {
sm: 'h-8 px-2 text-sm',
md: 'h-10 px-3 text-base',
lg: 'h-12 px-4 text-lg',
},
variant: {
default: 'border-gray-200 hover:border-gray-300',
ghost: 'border-transparent hover:bg-gray-100',
},
},
defaultVariants: {
size: 'md',
variant: 'default',
},
}
);

```

CSS-in-JS (Vanilla Extract, Stitches)

CSS-in-JS giver type-safe styling med tokens direkte i TypeScript. Vanilla Extract er populært fordi det kompilerer til statisk CSS — ingen runtime cost. Stitches er et andet godt valg med variant-support built-in.

Tip: CSS Modules + CSS custom properties er den mest portable løsning — den virker med alle frameworks og har ingen runtime overhead. Tailwind + CVA er godt hvis hele organisationen allerede bruger Tailwind.

8. Dokumentation med Storybook

Storybook er standarden for design system-dokumentation. Hver komponent får stories der viser alle varianter, states og edge cases.

```
// Select.stories.tsx
import type { Meta, StoryObj } from '@storybook/react';
import { Select } from '.';

const meta: Meta = {
  title: 'Components/Select',
  tags: ['autodocs'],
};
export default meta;

export const Default: StoryObj = {
  render: () => (
    <Select.Root>
    <Select.Trigger>
    <Select.Value placeholder="Vaelg..." />
    </Select.Trigger>
    <Select.Content>
    <Select.Item value="1">Option 1</Select.Item>
    <Select.Item value="2">Option 2</Select.Item>
    </Select.Content>
    </Select.Root>
  ),
};

export const Sizes: StoryObj = {
  render: () => (
    <div style={{ display: 'flex', gap: 16 }}>
    {[ 'sm', 'md', 'lg' ] as const}.map(size => (
    <Select.Root key={size}>
    <Select.Trigger size={size}>
    <Select.Value placeholder={size} />
    </Select.Trigger>
    <Select.Content>
    <Select.Item value="a">Item</Select.Item>
    </Select.Content>
    </Select.Root>
    )))
    </div>
  ),
};
```

9. Versionering og distribution

Package setup

```
// package.json
{
  "name": "@azets/ui",
  "version": "1.4.0",
  "main": "dist/index.js",
  "module": "dist/index.mjs",
  "types": "dist/index.d.ts",
  "exports": {
    ".": {
      "import": "./dist/index.mjs",
      "require": "./dist/index.js",
      "types": "./dist/index.d.ts"
    },
    "./tokens": {
      "import": "./dist/tokens/index.mjs",
      "types": "./dist/tokens/index.d.ts"
    }
  },
  "sideEffects": false,
  "peerDependencies": {
    "react": ">=18",
    "react-dom": ">=18"
  }
}
```

Semver-regler for design systems:

- **Patch (1.4.1)** — bugfixes, styling tweaks der ikke ændrer API
- **Minor (1.5.0)** — nye komponenter, nye props, nye varianter
- **Major (2.0.0)** — fjernede props, renamed komponenter, ændret token-struktur

Changesets til versionering:

```
# Installer changesets
npx @changesets/cli init

# Naar du laver en aendring
npx changeset
# > Vaelg package: @azets/ui
# > Patch / Minor / Major?
# > Beskriv aendringen

# I CI: version bump + publish
npx changeset version
npm publish
```

10. Governance og skalering

Når flere teams bruger dit design system, skal der være klare processer for at undgå kaos:

Contribution model

- **Centralt team** — ét team ejer design systemet og reviewer alle PRs
- **Federated** — alle teams kan bidrage, centralt team reviewer og godkender
- **Hybrid** — core components ejes centralt, domæne-specifikke komponenter ejes af teams

Regler for nye komponenter:

- Skal bruges af mindst 2 teams/produkter for at komme i core
- RFC/proposal først — beskriv API og use cases før implementering
- Accessibility review er mandatory
- Storybook documentation er mandatory
- Breaking changes kræver migration guide

11. Anti-patterns at undgå

Anti-pattern	Problem	Løsning
Prop explosion	30+ props, umuligt at vedligeholde	Compound components
God component	Ét komponent der kan alt	Split i fokuserede dele
Tight coupling til styling	Komponenter hardcoder farver	Design tokens + CSS vars
Ingen types	Props er any/untyped	Strikt TypeScript interfaces
Direkte Radix import i app	Mister kontrol over API	Wrap i eget namespace
Ingen docs	Ingen ved hvordan komponenter bruges	Storybook + autodocs
Copy-paste mellem teams	Divergerer over tid	Ét shared package

12. Checkliste

Brug denne checkliste når du starter eller evaluerer et design system:

- Design tokens er defineret centralt (farver, spacing, typografi, radii)
- Tokens bruger semantiske navne (colorPrimary, ikke indigo600)
- Komponenter bruger compound component pattern
- Radix (eller anden headless lib) wrapper er isoleret i eget namespace
- Forbrugere importerer aldrig direkte fra Radix
- Alle komponenter bruger forwardRef
- TypeScript interfaces for alle component props
- Storybook stories for hver komponent og variant
- Package.json har korrekte exports, types og peerDependencies
- Semver følges konsistent med changesets
- Contribution guidelines og RFC-proces er dokumenteret
- Accessibility: keyboard nav og ARIA er testet
- CI kører lint, typecheck, tests og Storybook build